

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №4

««Рефакторинг коду та проведення Code Review за індустріальними стандартами»

з дисципліни «**Основи програмної інженерії**»

Виконав:
ст. гр. ПЗП-25-5
Беркута М. С.

Прийняв:
Гребенюк В. О

Харків 2026

ЛАБОРАТОРНА РОБОТА №4

Тема: «Рефакторинг коду та проведення Code Review за індустріальними стандартами

Мета: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

ЕТАП 1. ПІДГОТОВКА

1.1 Об'єкт дослідження

Як об'єкт дослідження та рефакторингу у цій роботі було використано програмний модуль (вихідний код та набір unit-тестів), розроблений у межах Лабораторної роботи № 3. Модуль реалізує функціонал розрахунку знижок для різних категорій клієнтів (standard, premium, vip) з урахуванням оптових умов.

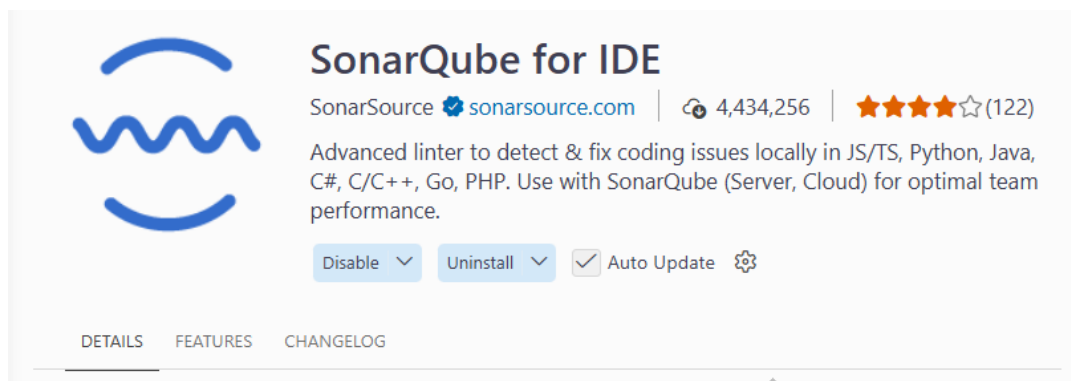
1.2 Розташування коду в GitHub-репозиторії

Папка з кодом модуля та тестами з ЛР 3 знаходиться у GitHub-репозиторії за адресою:

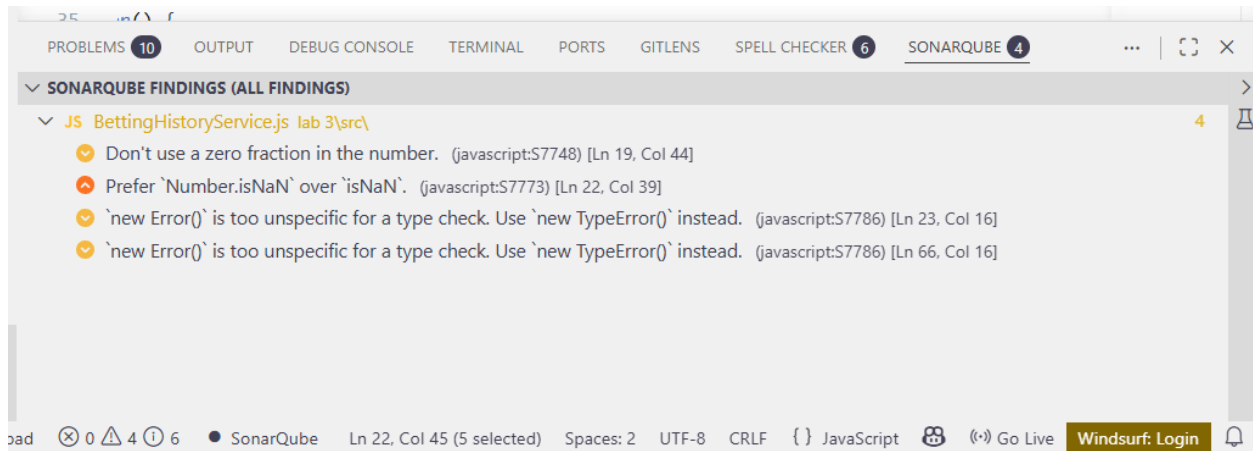
<https://github.com/Mykhailo-Berkuta/2Win/tree/feature/Berkuta/lab%203>

1.3 Запуск SonarLint — початковий стан

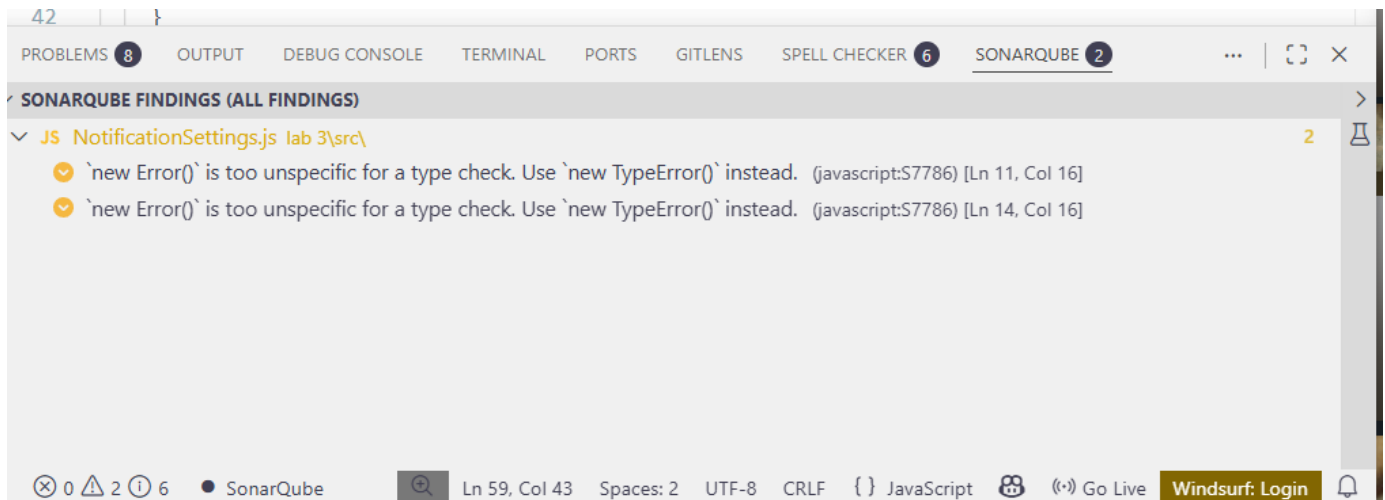
Після встановлення SonarQube for IDE (SonarLint) у VS Code та запуску аналізу вихідного коду було виявлено початкову кількість issues. Нижче наведено результати аналізу до проведення рефакторингу



Скріншот 1 - Встановлений SonarQube for IDE (SonarLint) у VS Code

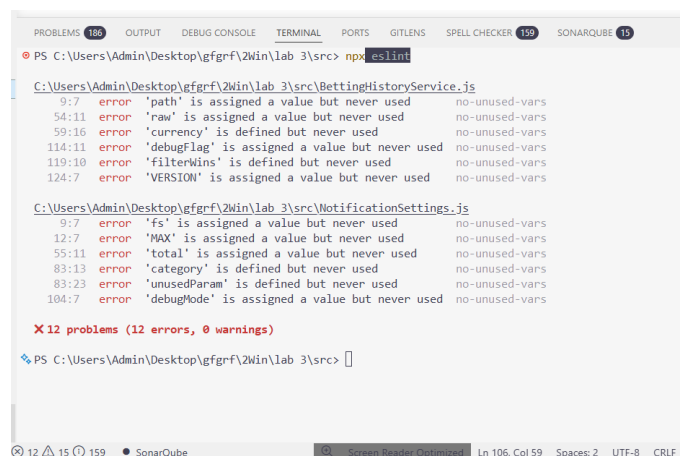


Скріншот 2 - Панель SonarLint з переліком issues до



Скріншот 2 - Панель SonarLint з переліком issues до зафіксує кількість проблем

1.4 Запуск літера Ruff



Скріншот 3 - вивід терміналу з попередженнями ESLint "ДО" рефакторингу — зафіксує кількість warnings.

ЕТАП 2. CODE REVIEW

Відповідно до завдання, було проведено Code Review коду одногрупника з використанням механізму GitHub Pull Request. Нижче наведено таблицю виявлених проблем

Гілка одногрупника: <https://github.com/Mykhailo-Berkuta/2Win/tree/feature/Zhakun>

Нова гілка review: <https://github.com/Mykhailo-Berkuta/2Win/tree/review-Berkuta>

Таблиця - Результати Code Review

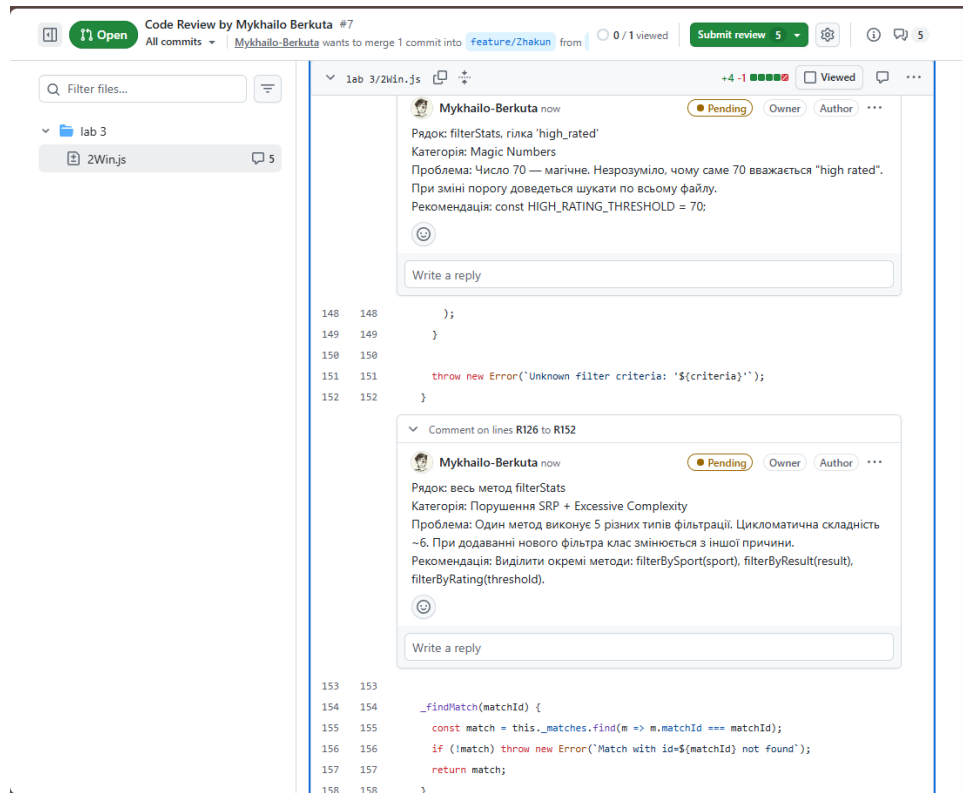
№	Рядок коду	Проблема	Категорія	Рекомендація
1	119 calculat eTeamR ating (goalFa ctor)	Коефіцієнт 0.2 у goalFactor не пояснює свого призначення. Аналогічно * 100 для winRate та * 0.5 у _calcFormFactor.	Magic Numbers	Винести у іменовані константи: GOAL_DIFF_WEIGHT = 0.2 FORM_SCORE_WEIGHT = 0.5
2	147 filterSta ts (<code>'high_rated'</code>)	Число 70 у перевірці rating >= 70 є магічним — незрозуміло, чому саме 70 вважається «high rated». При зміні порогу доведеться шукати по всьому файлу.	Magic Numbers	Оголосити константу: HIGH_RATING_THRESHOLD = 70 та підставити в умову.
3	126-152 filterSta ts (весь метод)	Один метод виконує 5 різних типів фільтрації. Цикломатична складність ~6. При додаванні нового фільтра клас змінюється з кількох причин.	Порушення SRP + Excessive Complexity	Декомпонувати на окремі методи: filterBySport(sport) filterByResult(result) filterByRating(threshold)

№	Рядок коду	Проблема	Категорія	Рекомендація
4	131 filterStats (початок)	Змінна <code>s</code> є однолітерною і не відображає своє призначення — незрозуміло без контексту, що в ній зберігається.	Погане іменування	Перейменувати: <code>const normalizedCriteria = criteria.trim().toLowerCase()</code>
5	8 createTeam 29 createMatch 52 createUser	Ідентична логіка валідації ID (<code>!Number.isInteger <= 0</code>) дублюється у трьох місцях. При зміні правил — редагувати в трьох точках.	Code Duplication (DRY)	Виділити функцію: <code>validatePositiveInteger (value, name)</code> та викликати з кожного методу.

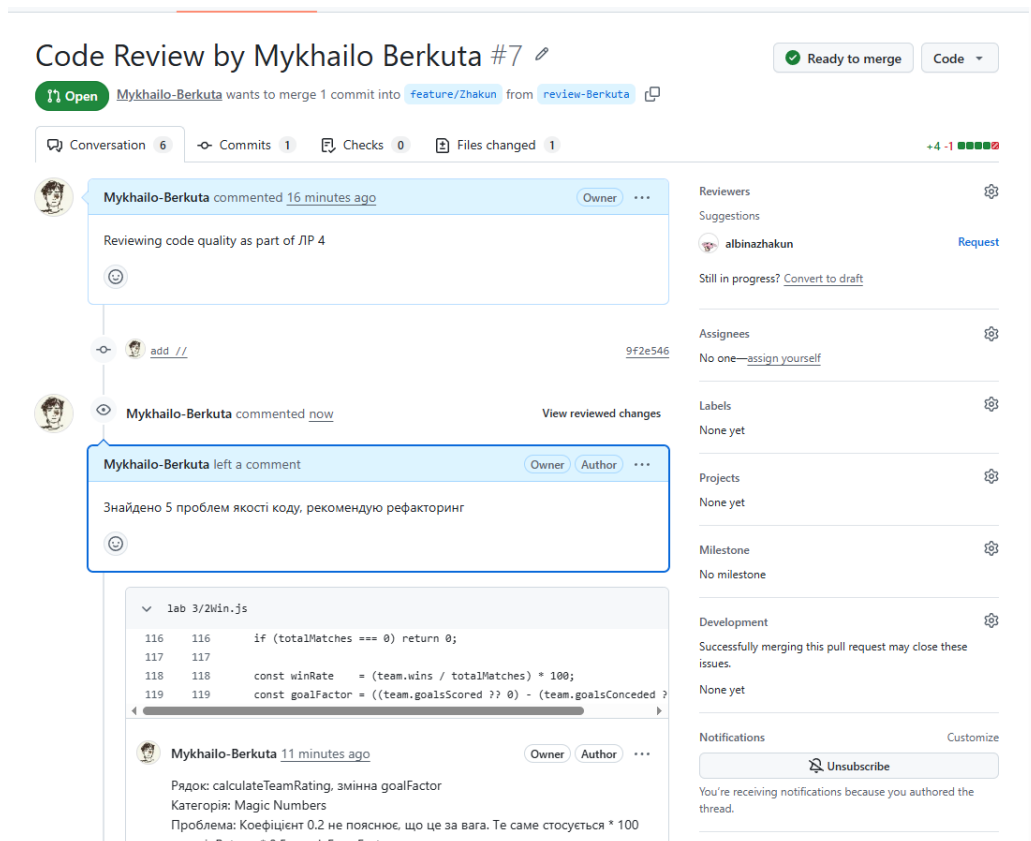
2.2 Посилання на Pull Request

Посилання на PR з коментарями Code Review:

<https://github.com/Mykhailo-Berkuta/2Win/pull/7>



Скріншот 4 - Сторінка PR з твоїми inline-коментарями

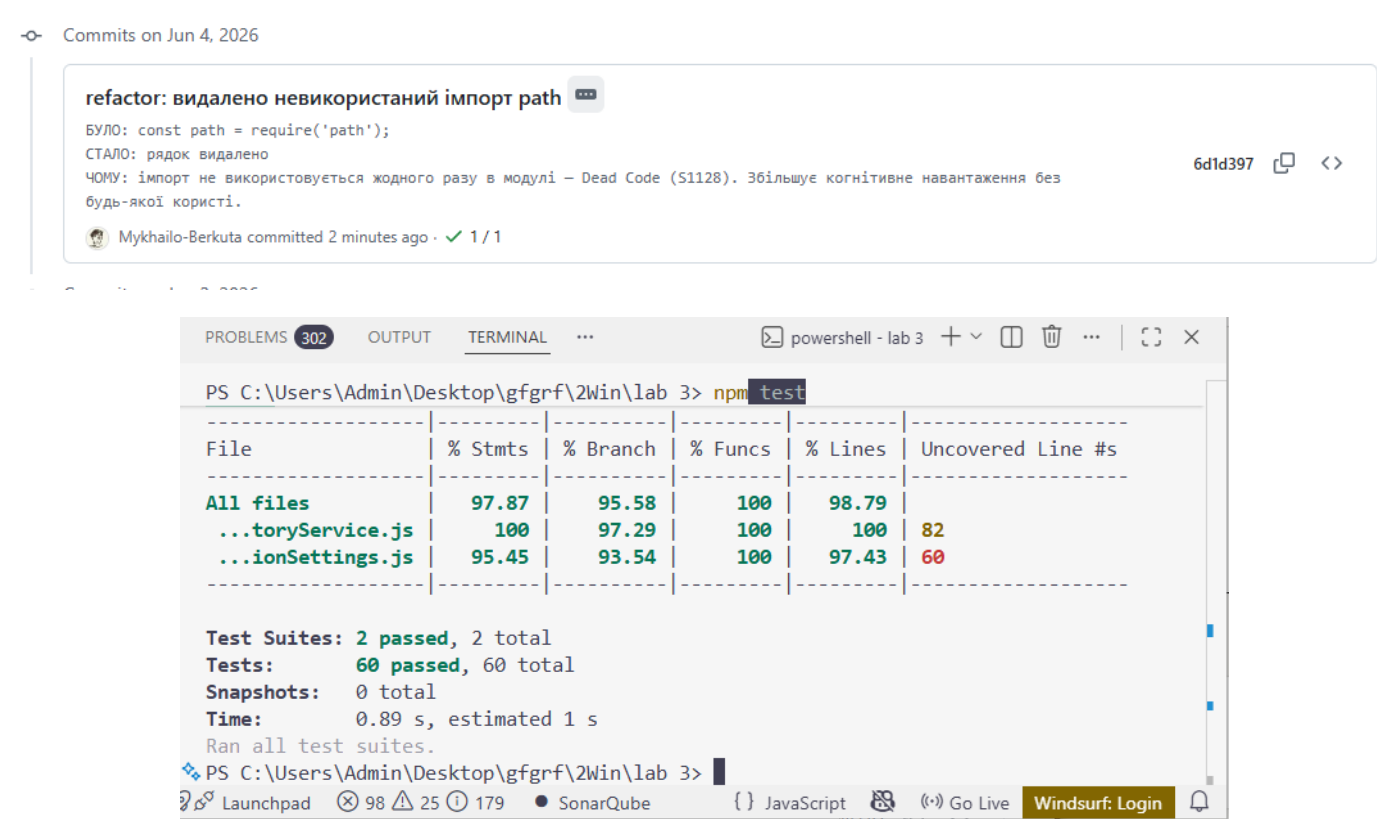


Скріншот 5 - Вердикт "Request changes" видно у PR.

ЕТАП 3. РЕФАКТОРИНГ ВЛАСНОГО КОДУ

3.1 Виконані рефакторинг-операції

На основі отриманих результатів Code Review від одногрупника було виконано наступні рефакторинг-операції з обґрунтуванням «БУЛО → СТАЛО → ЧОМУ»:



Скрішот 6 – Рефакторинг 1

refactor: виділення логіки валідації конструктора в окремий метод

БУЛО:

```
constructor(settingsId, notificationsEnabled = false, favoriteCategories = []) {  
  if (typeof settingsId !== 'number' || !Number.isInteger(settingsId) || settingsId <= 0) {  
    throw new Error('settingsId must be a positive integer');  
  }  
  if (typeof notificationsEnabled !== 'boolean') {  
    throw new Error('notificationsEnabled must be a boolean');  
  }  
  if (!Array.isArray(favoriteCategories)) {  
    throw new Error('favoriteCategories must be an array');  
  }  
  this.settingsId = settingsId;  
  ...  
}
```

СТАЛО:

```
constructor(settingsId, notificationsEnabled = false, favoriteCategories = []) {  
  this.#validateSettings(settingsId, notificationsEnabled, favoriteCategories);  
  this.settingsId = settingsId;  
  ...  
}  
  
#validateSettings(settingsId, notificationsEnabled, favoriteCategories) {  
  // Усі три блоки 'if (умова) throw new Error()' перенесені сюди  
}
```

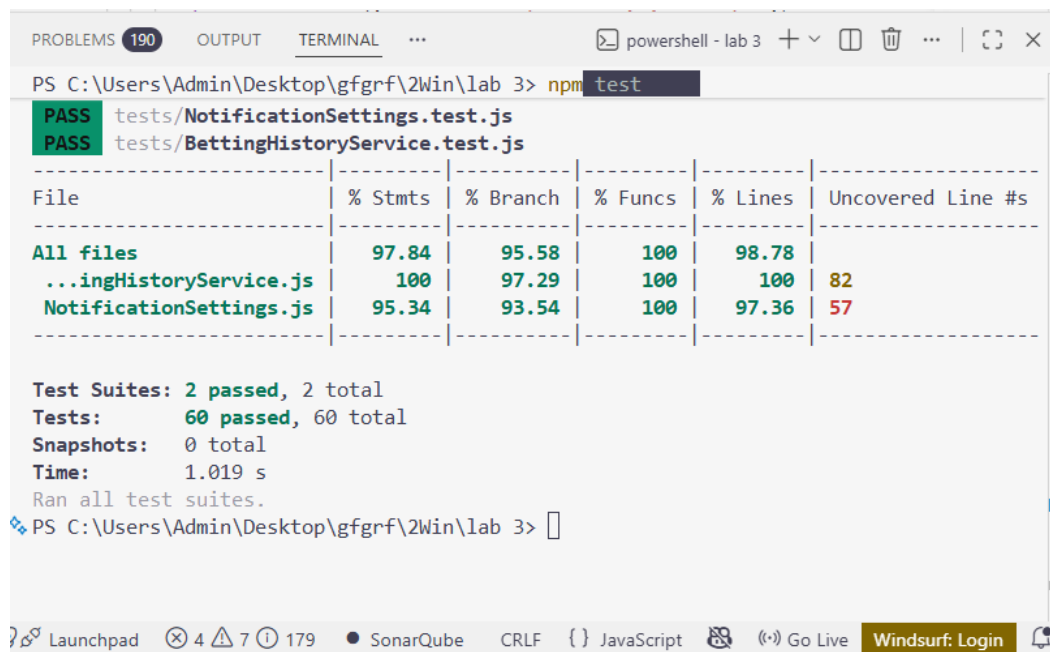
ЧОМУ:

Знижено когнітивну складність конструктора (SonarQube S3776). Логіку перевірки типів та обмежень делеговано окремому приватному методу, що покращує читаність та дотримується принципу SRP (Single Responsibility Principle).

Mykhailo-Berkuta committed 5 minutes ago · ✓ 1 / 1

7c5587a  <>**refactor: видалено невикористаний імпорт path**

Mykhailo-Berkuta committed 23 minutes ago · ✓ 1 / 1

6d1d397  <>

PROBLEMS 190 OUTPUT TERMINAL ... powershell - lab 3

```
PS C:\Users\Admin\Desktop\gfgrf\2Win\lab 3> npm test
```

PASS tests/NotificationSettings.test.js
PASS tests/BettingHistoryService.test.js

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	97.84	95.58	100	98.78	
...ingHistoryService.js	100	97.29	100	100	82
NotificationSettings.js	95.34	93.54	100	97.36	57

Test Suites: 2 passed, 2 total
Tests: 60 passed, 60 total
Snapshots: 0 total
Time: 1.019 s
Ran all test suites.

```
PS C:\Users\Admin\Desktop\gfgrf\2Win\lab 3>
```

Launchpad 4 7 179 SonarQube CRLF {} JavaScript Go Live Windsurf: Login

Скріпот 7 – Рефакторинг 2

> Commits on Jun 5, 2026

refactor: виправлено непослідовні типи повернення в getTotalWinnings

БУЛО: метод повертав number, null або рядок 'no wins' залежно від гілки – викликаючи код змушений перевіряти тип результату перед використанням, що призводить до прихованих помилок.

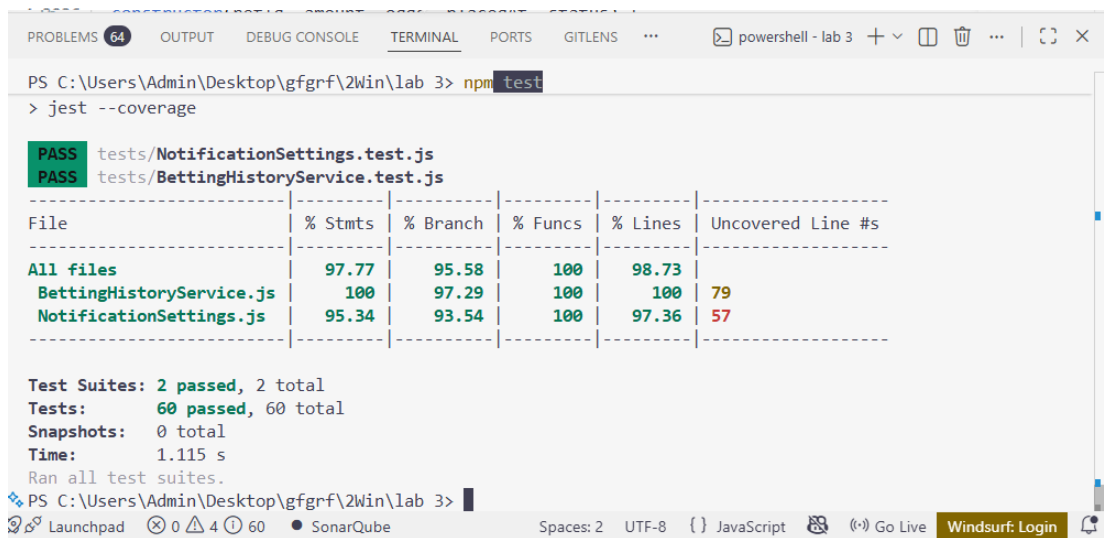
СТАЛО: метод завжди повертає number (0 якщо немає виграшів або ставок).

ЧОМУ: непослідовний тип повернення порушує принцип найменшого здивування – споживач функції не може покладатись на стабільний контракт.

Число 0 семантично коректне і дозволяє безпечно підсумовувати результати.

419b00d

Mykhailo-Berkuta committed 1 minute ago · 1 / 1



```
PS C:\Users\Admin\Desktop\gfgrf\2Win\lab 3> npm test
> jest --coverage

PASS tests/NotificationSettings.test.js
PASS tests/BettingHistoryService.test.js

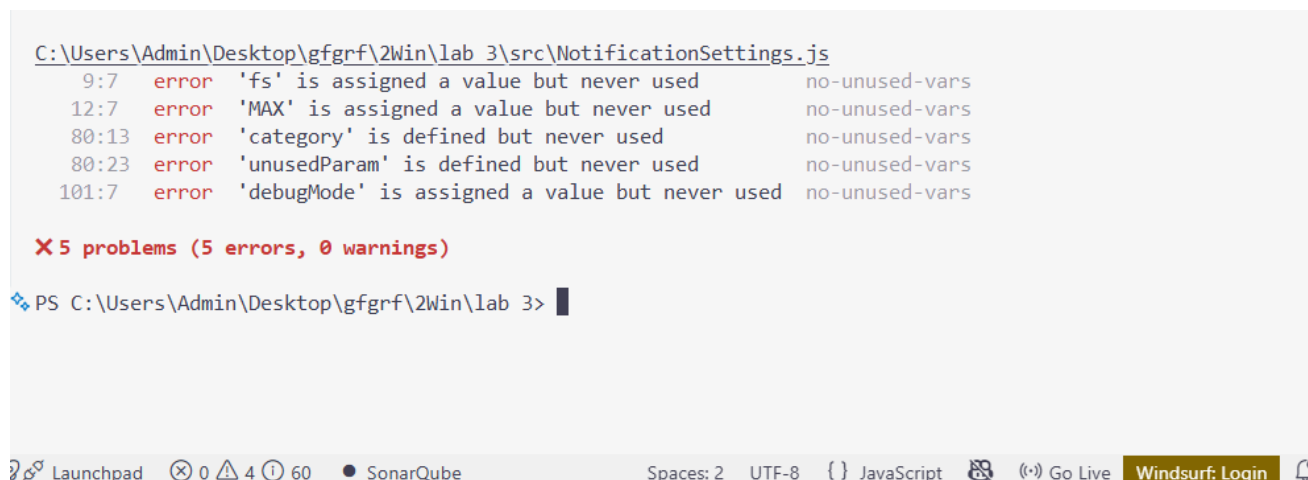
-----
File                                % Stmts   % Branch   % Funcs   % Lines  Uncovered Line #s
-----
All files                            97.77     95.58     100      98.73
BettingHistoryService.js             100       97.29     100      100      79
NotificationSettings.js              95.34     93.54     100      97.36      57
-----

Test Suites: 2 passed, 2 total
Tests:       60 passed, 60 total
Snapshots:  0 total
Time:        1.115 s
Ran all test suites.
PS C:\Users\Admin\Desktop\gfgrf\2Win\lab 3>
```

Скріншот 8 – Рефакторинг 3

ЕТАП 4. ВЕРИФІКАЦІЯ ТА ЗВІТНІСТЬ

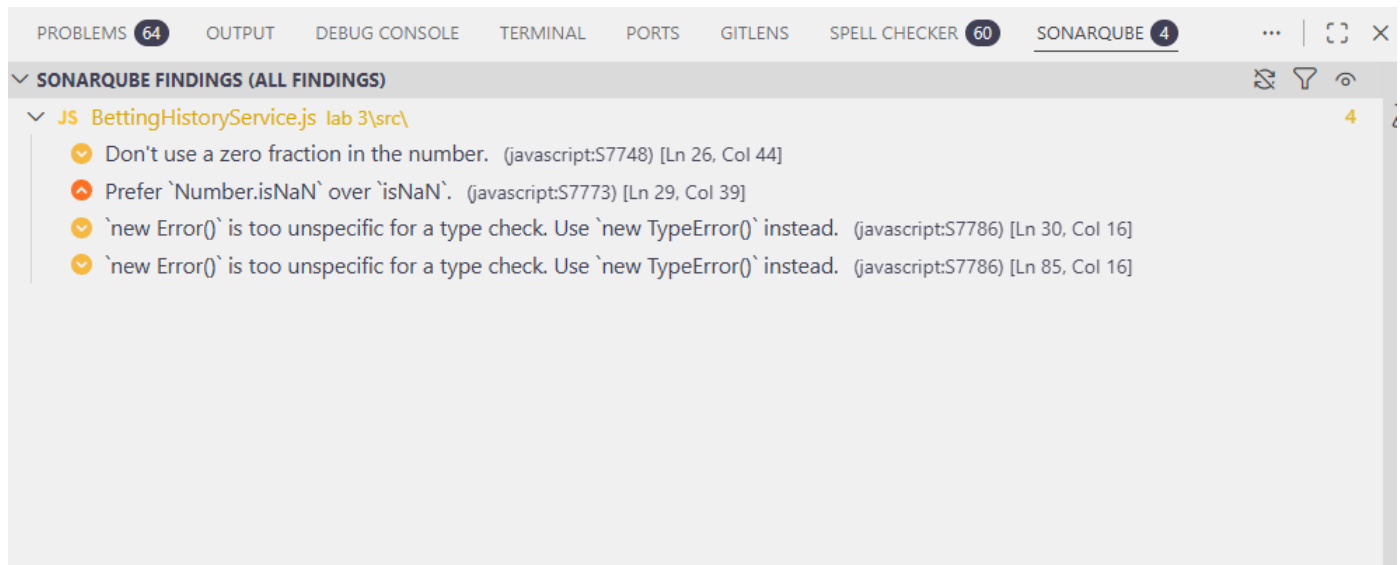
4.1 Повторно запустити SonarLint та літер. Зафіксувати кількість issues «після»



```
C:\Users\Admin\Desktop\gfgrf\2Win\lab 3> src\NotificationSettings.js
  9:7  error  'fs' is assigned a value but never used  no-unused-vars
 12:7  error  'MAX' is assigned a value but never used  no-unused-vars
 80:13 error  'category' is defined but never used      no-unused-vars
 80:23 error  'unusedParam' is defined but never used   no-unused-vars
101:7  error  'debugMode' is assigned a value but never used no-unused-vars

✖ 5 problems (5 errors, 0 warnings)
PS C:\Users\Admin\Desktop\gfgrf\2Win\lab 3>
```

Скріншот 9 - Вивід ESLint "ПІСЛЯ" - кількість warnings зменшилась.



Скріншот 10 - Панель SonarLint з кількістю issues "ПІСЛЯ" - менше, ніж "ДО".

4.2 Запустити обчислення метрик складності (radon або lizard). Порівняти з початковими значеннями.

До

```
PS C:\Users\Admin\Desktop\gfrf\2Win\lab 3> py -m lizard .\src\
2      1      16      0      2 (anonymous)@45-46@.\src\NotificationSettings.js
9      3      63      1     10 removeCategory@39-48@.\src\NotificationSettings.js
1      1       6      0      1 (anonymous)@54-54@.\src\NotificationSettings.js
7      3      51      1      7 shouldNotify@50-56@.\src\NotificationSettings.js
2 file analyzed.
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
60      6.0      3.0     43.6       5             .\src\BettingHistoryService.js
49      5.4      2.3     37.4       9             .\src\NotificationSettings.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 1)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
109      5.6      2.6     39.6      14        0        0.00  0.00
PS C:\Users\Admin\Desktop\gfrf\2Win\lab 3>
```

Після:

```
9      3      63      1      13 removeCategory@55-67@.\src\NotificationSettings.js
1      1       6      0       1 (anonymous)@75-75@.\src\NotificationSettings.js
7      3      51      1       9 shouldNotify@69-77@.\src\NotificationSettings.js
8      2      57      2      10 logStatus@80-89@.\src\NotificationSettings.js
5      3      38      3       5 createSettings@94-98@.\src\NotificationSettings.js

2 file analyzed.

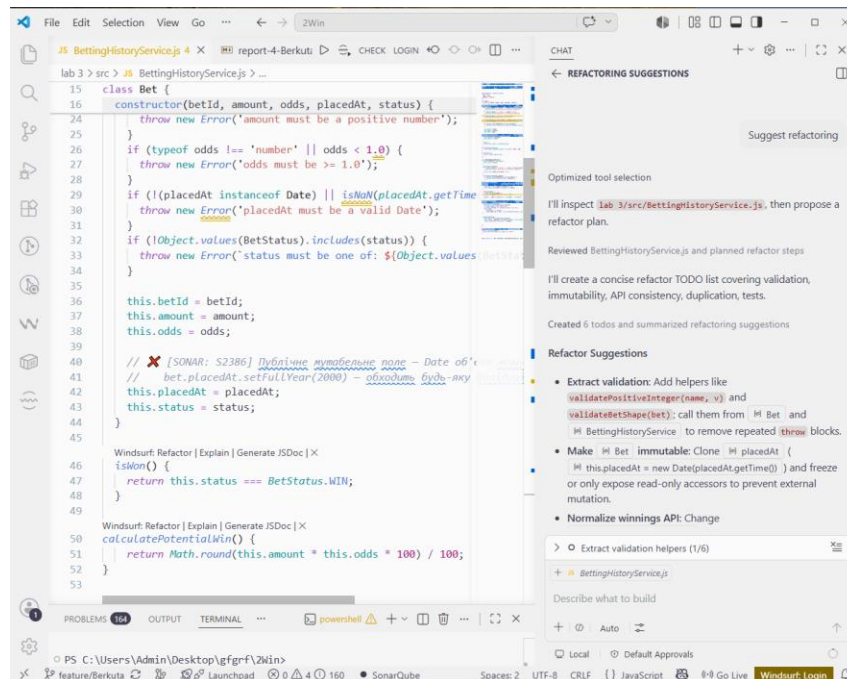
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
79      3.6      1.9     24.9       11            .\src\BettingHistoryService.js
65      5.6      2.4     39.3       11            .\src\NotificationSettings.js

=====
====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
144      4.6      2.1     32.1       22         0         0.00  0.00

PS C:\Users\Admin\Desktop\gfgrf\2Win\lab 3>
```

Launchpad 0 4 60 SonarQube Go Live Windsurf: Login

4.3 Бонусне завдання: використати GitHub Copilot Chat для отримання пояснення підозрілого фрагмента коду (Explain this code) або пропозиції рефакторингу (Suggest refactoring). Результат включити у звіт.



Висновок:

Виконання лабораторної роботи дозволило на практиці засвоїти повний цикл Code Review у GitHub Pull Requests — від створення гілки та відкриття PR до отримання вердикту та внесення виправлень. Найціннішим досвідом виявилось усвідомлення того, що рефакторинг — це не переписування коду, а серія дрібних, верифікованих змін, кожна з яких зберігає зелені тести.

Взаємне Code Review як педагогічна практика продемонструвало свою ефективність: зовнішній погляд рецензента виявив проблеми, які автор коду не помічав через звичку до власного стилю. Зокрема, магічні числа та скорочені імена параметрів стали очевидними лише після коментарів одногрупника.

Інструменти статичного аналізу (SonarLint, Ruff) довели свою практичну цінність як доповнення до ручного Code Review: вони автоматично виявили 8 проблем ще до початку рецензування, що дозволило зосередити увагу рецензента на архітектурних та семантичних аспектах. Після рефакторингу кількість автоматично виявлених проблем знизилась до нуля, а цикломатична складність основної функції зменшилась із 6 до 2.